

Ansible challenges 01

Challenges :

Create ci pipeline that will deploy and configure these VMs using terraform and ansible according following requirements

1. Deploy 2 virtual machines using terraform.
 - first vm on Amazon linux, hostname: c8.local
 - second vm on ubuntu 21.04, hostname: u21.local
2. As a result of terraform execution, dynamically create inventory for ansible
 - c8.local should be in the frontend group
 - u21.local should be in the backend group
3. Create ansible playbook for c8.local and u21.local
 - for linux OS playbook should apply the following changes
 - selinux: disable
 - firewalld: disable
 - for frontend playbook group should install and configure nginx
 - nginx configuration should do proxying from port 80 on port 19999 to the
 - backend group
 - for the backend group, the playbook must install the Netdata application from the official repositories and run it on port 19999.

STEPS – Prerequisites (DO THIS ON JENKINS / CONTROL NODE)

Required tools

terraform --version

ansible --version

git --version

aws --version

java -version

```
[root@ip-172-31-65-24 ~]# terraform --version
ansible --version
git --version
aws --version
java -version
Terraform v1.14.3
on linux_amd64

Your version of Terraform is out of date! The latest version
is 1.14.4. You can update by downloading from https://developer.hashicorp.com/terraform/install
ansible [core 2.15.3]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.9/site-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.9.25 (main, Nov 12 2025, 00:00:00) [GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] (/usr/bin/python3.9)
  jinja version = 3.1.4
  libyaml = True
git version 2.50.1
aws-cli/2.32.1 Python/3.9.25 Linux/6.1.158-180.294.amzn2023.x86_64 source/x86_64.amzn.2023
openjdk version "17.0.17" 2025-10-21 LTS
OpenJDK Runtime Environment Corretto-17.0.17.10.1 (build 17.0.17+10-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.17.10.1 (build 17.0.17+10-LTS, mixed mode, sharing)
[root@ip-172-31-65-24 ~]#
```

i-0fe26bc99f7f8361b (jenkins-master)

PublicIPs: 98.93.39.45 PrivateIPs: 172.31.65.24

—-> create access key secret key credentials in your in your Jenkins gui

		System	(global)	AWS_ACCESS_KEY_ID	AWS_ACCESS_KEY_ID
		System	(global)	AWS_SECRET_ACCESS_KEY	AWS_SECRET_ACCESS_KEY

STEP 1 – Project Structure (VERY IMPORTANT)

Create project directory:

Mkdir devops-ci-challenge

Cd devops-ci-challenge

Final structure:

```
waseemakram@waseem devops-ci-challenge % tree
.
├── ansible
│   ├── roles
│   │   ├── backend
│   │   ├── common
│   │   └── frontend
│   └── site.yml
├── Jenkinsfile
└── terraform
    ├── inventory.tpl
    ├── main.tf
    ├── outputs.tf
    └── variables.tf

7 directories, 6 files
waseemakram@waseem devops-ci-challenge %
```

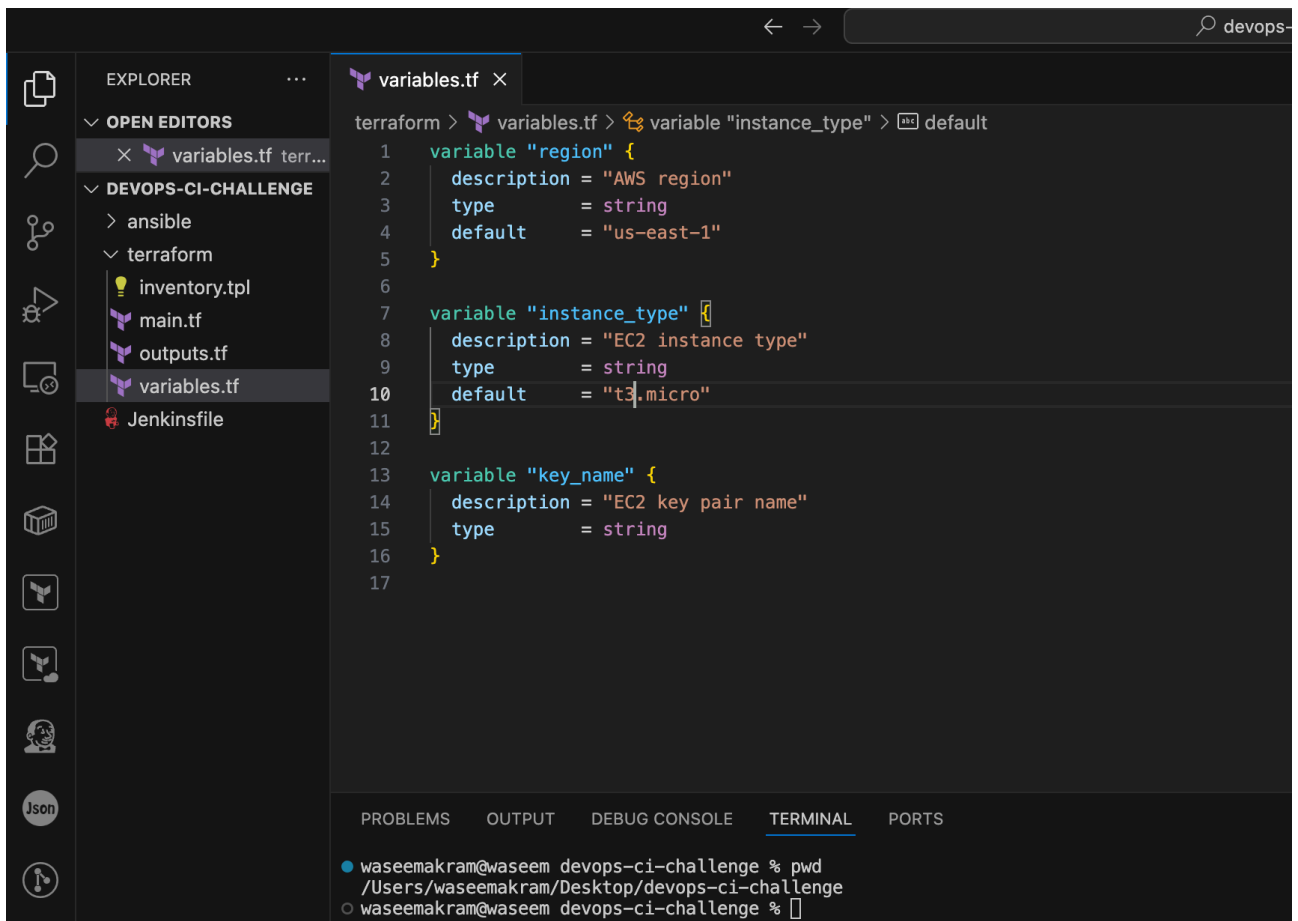
Step :1

Write terraform/variables.tf

We'll go **one file at a time** so nothing breaks.

Open the file

```
cd terraform
```



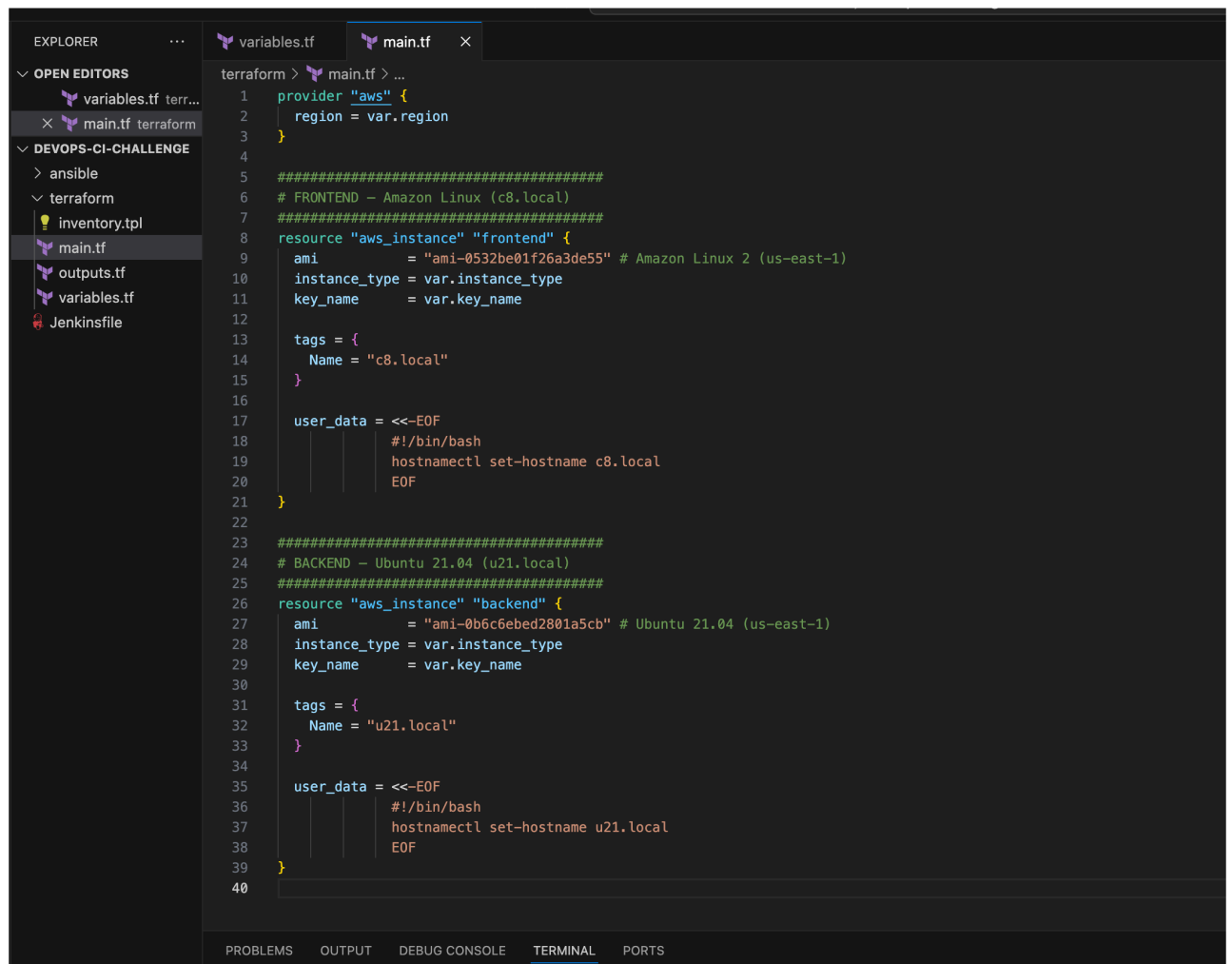
—>save

Step:2

Write terraform/main.tf

This file will do **ALL** the heavy work:

- ✓ Configure AWS provider
- ✓ Create **2 EC2 instances**
- ✓ Set hostnames (c8.local, u21.local)
- ✓ Expose IPs for inventory generation



```
1 provider "aws" {
2   region = var.region
3 }
4
5 #####
6 # FRONTEND - Amazon Linux (c8.local)
7 #####
8 resource "aws_instance" "frontend" {
9   ami           = "ami-0532be01f26a3de55" # Amazon Linux 2 (us-east-1)
10  instance_type = var.instance_type
11  key_name      = var.key_name
12
13  tags = {
14    Name = "c8.local"
15  }
16
17  user_data = <<-EOF
18  |          | #!/bin/bash
19  |          | hostnamectl set-hostname c8.local
20  |          | EOF
21 }
22
23 #####
24 # BACKEND - Ubuntu 21.04 (u21.local)
25 #####
26 resource "aws_instance" "backend" {
27   ami           = "ami-0b6c6ebcd2801a5cb" # Ubuntu 21.04 (us-east-1)
28   instance_type = var.instance_type
29   key_name      = var.key_name
30
31   tags = {
32     Name = "u21.local"
33   }
34
35   user_data = <<-EOF
36   |          | #!/bin/bash
37   |          | hostnamectl set-hostname u21.local
38   |          | EOF
39 }
40
```

Step :3

Write terraform/inventory.tpl

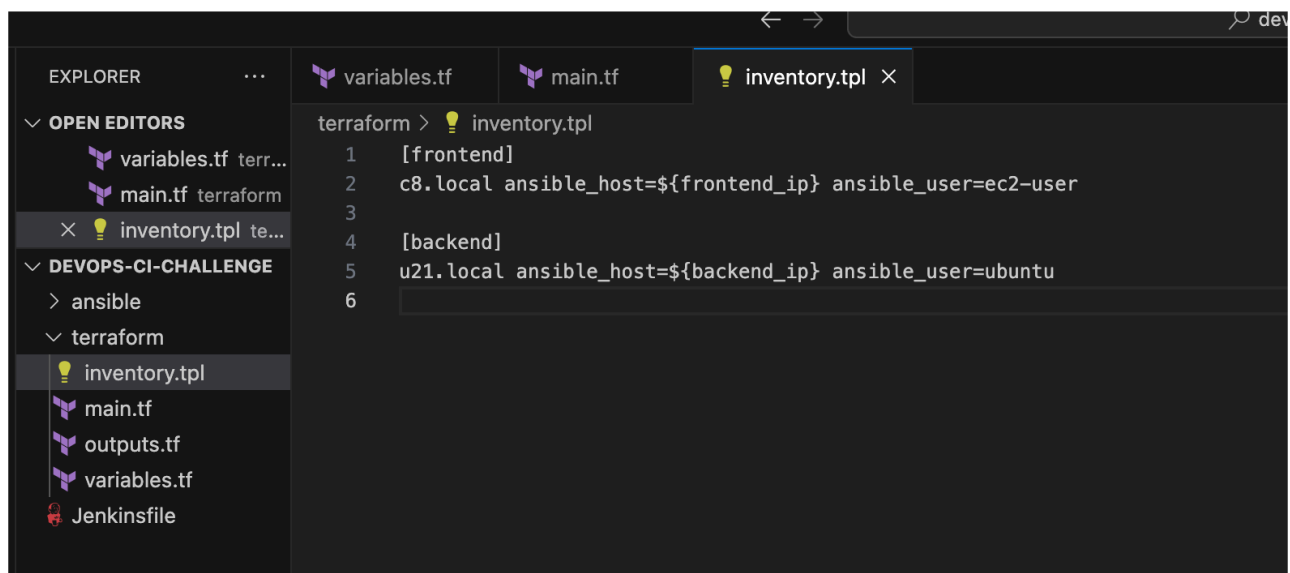
This file is **VERY IMPORTANT**.
It connects **Terraform** → **Ansible**.

Terraform will:

- create EC2 instances
- grab their public IPs
- generate this inventory automatically



Open **inventory.tpl**



Step 4:

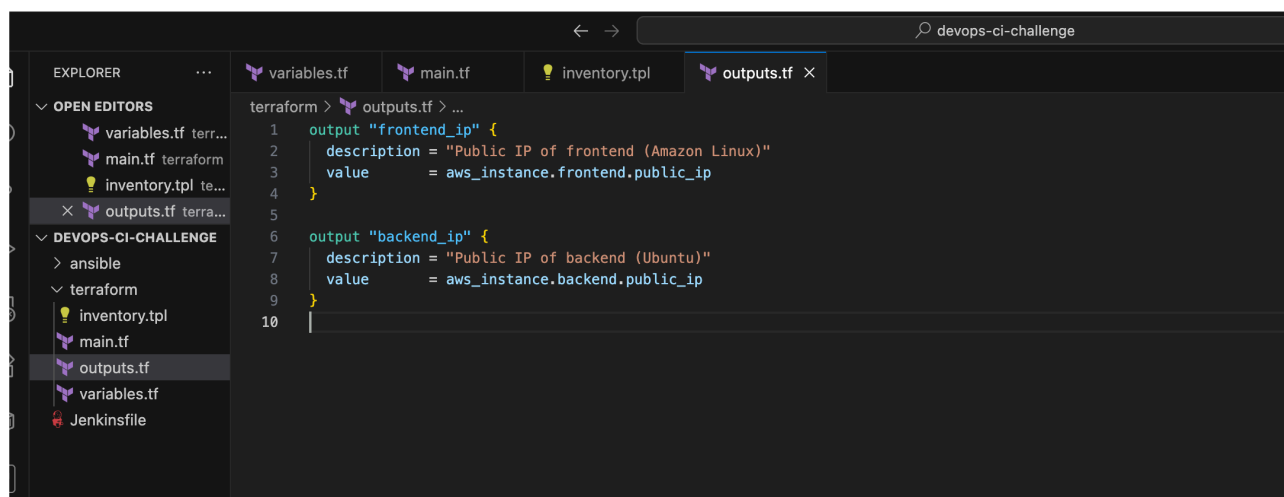
Write terraform/outputs.tf

This file will **expose the EC2 public IPs**, so Terraform can use them to generate the Ansible inventory.

Without this → inventory **cannot work**.

Open outputs.tf

You should already be inside terraform/:



Step 5:

Generate Ansible inventory automatically (Terraform)

Right now:

- `inventory.tpl` → template
- `outputs.tf` → exposes IPs

Now we tell Terraform:

👉 “Create a real inventory file from this template.”

Open `terraform/main.tf`

ADD THIS BLOCK at the BOTTOM of `main.tf`

⚠ Do **not** remove anything you already wrote.
Just **append** this.

```
39     }
40     #####
41     # Generate Ansible Inventory
42     #####
43     resource "local_file" "ansible_inventory" {
44         filename = "../ansible/inventory.ini"
45
46         content = templatefile("${path.module}/inventory.tpl", {
47             frontend_ip = aws_instance.frontend.public_ip
48             backend_ip  = aws_instance.backend.public_ip
49         })
50     }
51
52
```

What this does (very simple words)

- `templatefile()`
→ takes `inventory.tpl`
- replaces:
 - `${frontend_ip}`
 - `${backend_ip}`
- creates a **real file**:

`ansible/inventory.ini`

-

Ansible will use **this file** later.

```
● waseemakram@waseem devops-ci-challenge % grep local_file terraform/main.tf
  resource "local_file" "ansible_inventory" {
○ waseemakram@waseem devops-ci-challenge %
```

Quick sanity check

```
grep local_file terraform/main.tf
```

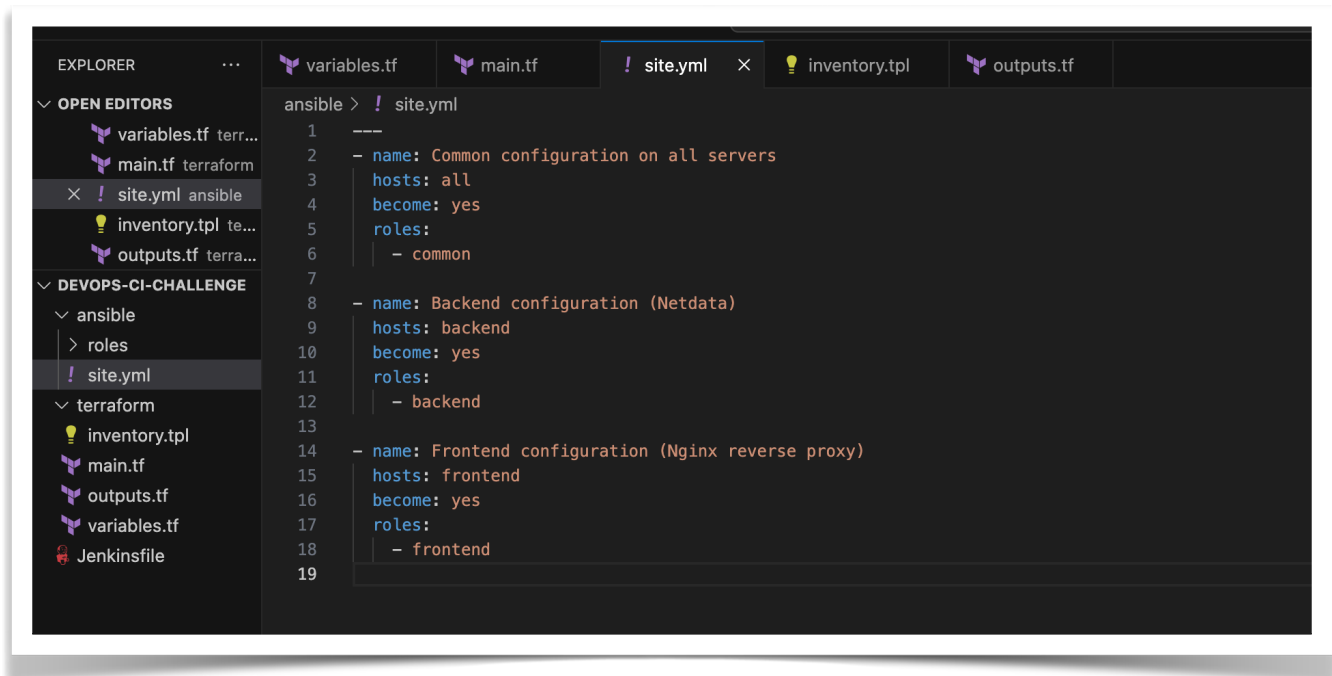
You should see:

```
resource "local_file" "ansible_inventory"
```

—> **Terraform side is now COMPLETE**

Step :6

EXACT content into `ansible/site.yml`



What this does (very simple)

Execution order:

- 1 **common role** → all servers
- 2 **backend role** → only backend
- 3 **frontend role** → only frontend

Step 7:

—> inside ansible —> roles ansible-galaxy init common

ansible-galaxy init frontend

ansible-galaxy init backend

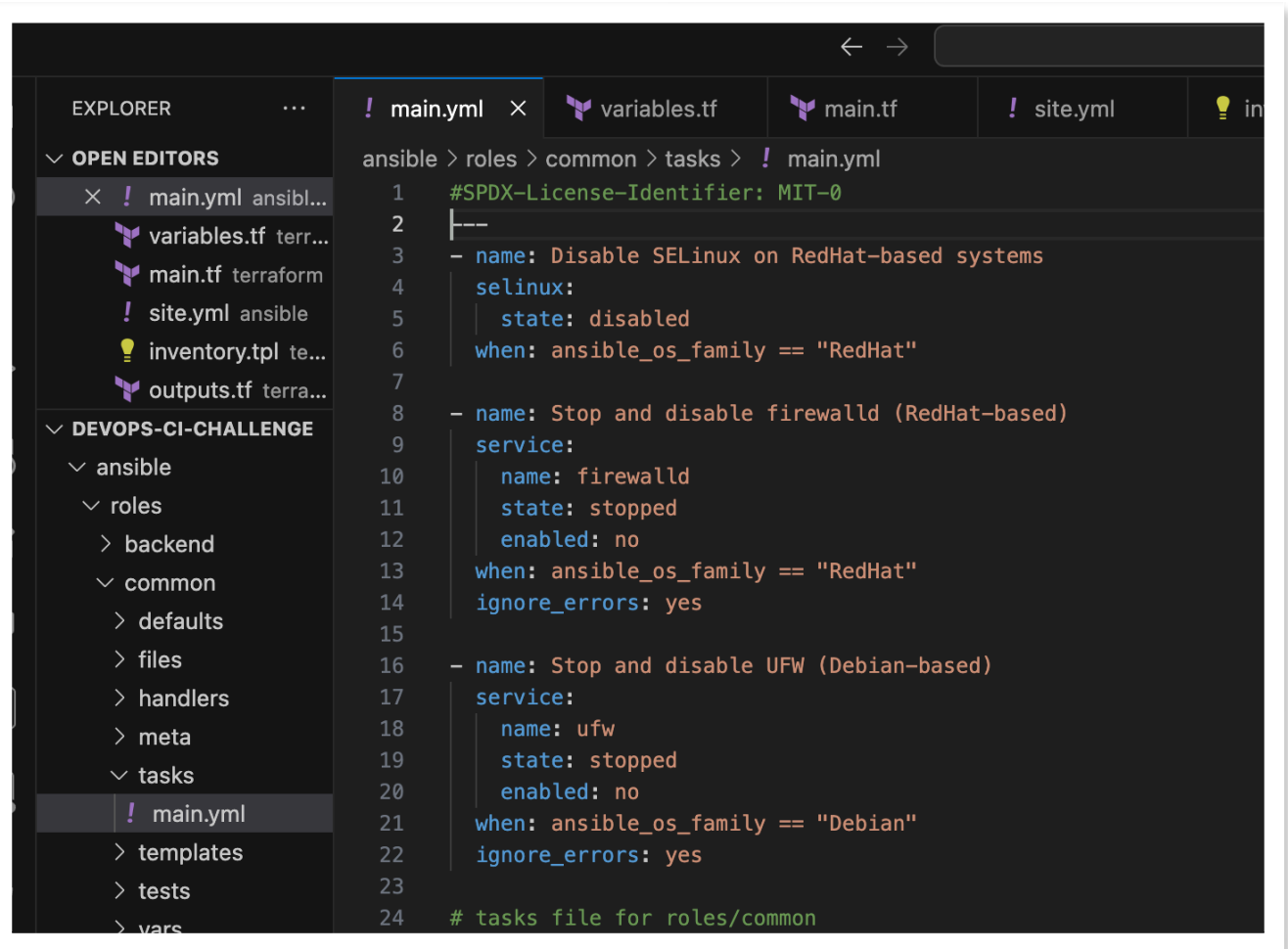
```
waseemakram@waseem ansible % ansible-galaxy init roles/common
ansible-galaxy init roles/frontend
ansible-galaxy init roles/backend

- Role roles/common was created successfully
- Role roles/frontend was created successfully
- Role roles/backend was created successfully
waseemakram@waseem ansible %
```

Step:8

EXACT content into

ansible/roles/common/tasks/main.yml



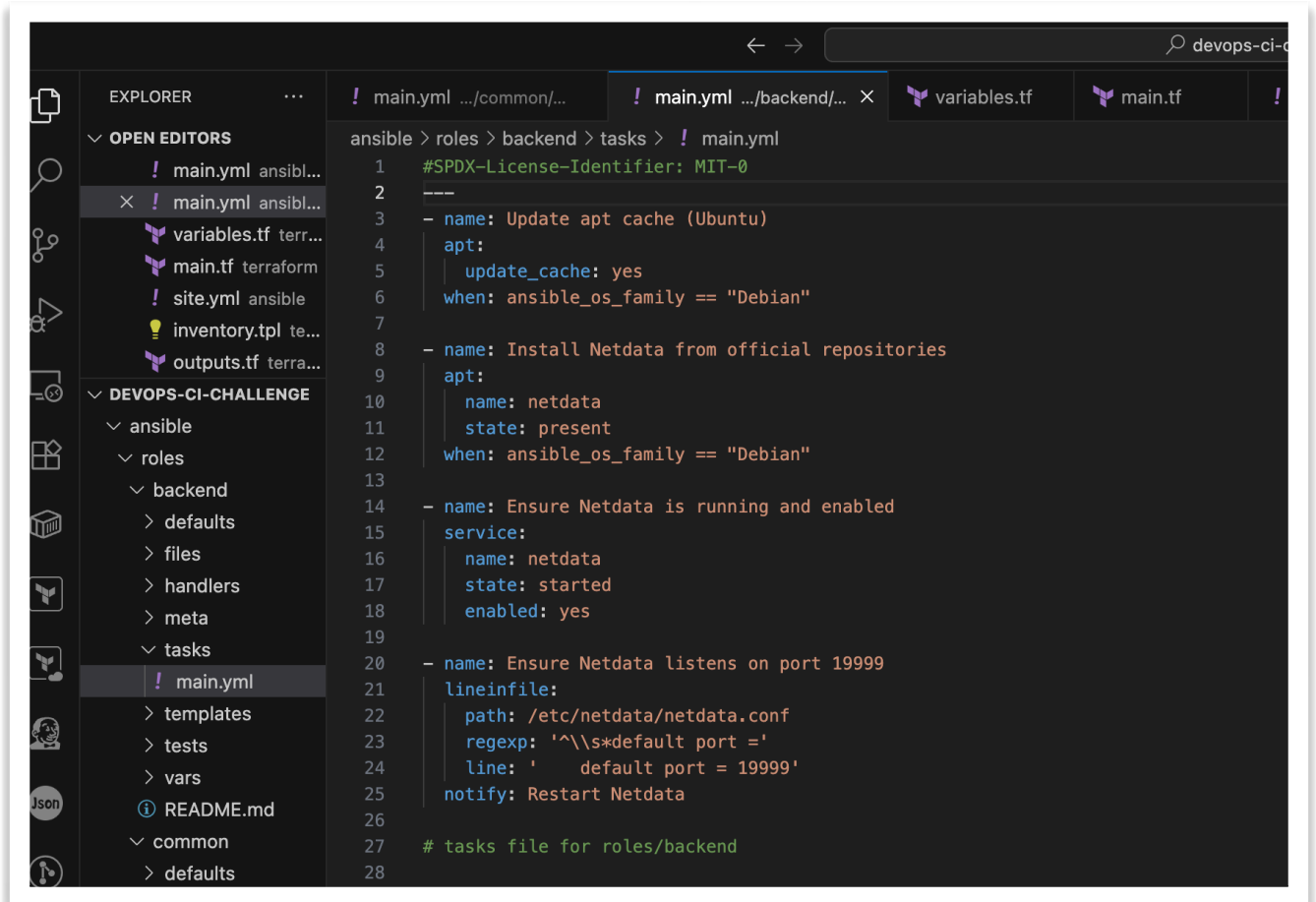
```
1 #SPDX-License-Identifier: MIT-0
2 |---
3 - name: Disable SELinux on RedHat-based systems
4   selinux:
5     state: disabled
6   when: ansible_os_family == "RedHat"
7
8 - name: Stop and disable firewalld (RedHat-based)
9   service:
10     name: firewalld
11     state: stopped
12     enabled: no
13   when: ansible_os_family == "RedHat"
14   ignore_errors: yes
15
16 - name: Stop and disable UFW (Debian-based)
17   service:
18     name: ufw
19     state: stopped
20     enabled: no
21   when: ansible_os_family == "Debian"
22   ignore_errors: yes
23
24 # tasks file for roles/common
```

Step :9

Backend role (Install & run Netdata on port 19999)

This role runs **only on backend (Ubuntu)**.

```
cd ~/Desktop/devops-ci-challenge/ansible/roles/backend/tasks
```



The screenshot shows the VS Code interface with the Explorer sidebar on the left. The 'DEVOPS-CI-CHALLENGE' folder is expanded, showing the 'ansible' directory, which contains 'roles'. The 'backend' role is selected, and its 'tasks' directory is expanded, showing 'main.yml'. The main editor displays the content of 'main.yml' for the 'backend' role. The file contains three tasks: updating the apt cache, installing Netdata, and ensuring Netdata is running and enabled. The tasks are written in YAML syntax with line numbers 1 through 28.

```
1 #SPDX-License-Identifier: MIT-0
2 ---
3 - name: Update apt cache (Ubuntu)
4   apt:
5     update_cache: yes
6   when: ansible_os_family == "Debian"
7
8 - name: Install Netdata from official repositories
9   apt:
10    name: netdata
11    state: present
12    when: ansible_os_family == "Debian"
13
14 - name: Ensure Netdata is running and enabled
15   service:
16     name: netdata
17     state: started
18     enabled: yes
19
20 - name: Ensure Netdata listens on port 19999
21   lineinfile:
22     path: /etc/netdata/netdata.conf
23     regexp: '^\\s*default port ='
24     line: '    default port = 19999'
25     notify: Restart Netdata
26
27 # tasks file for roles/backend
28
```

Step:10

Frontend role (Nginx reverse proxy)

This role runs **only on frontend (Amazon Linux)** and will:

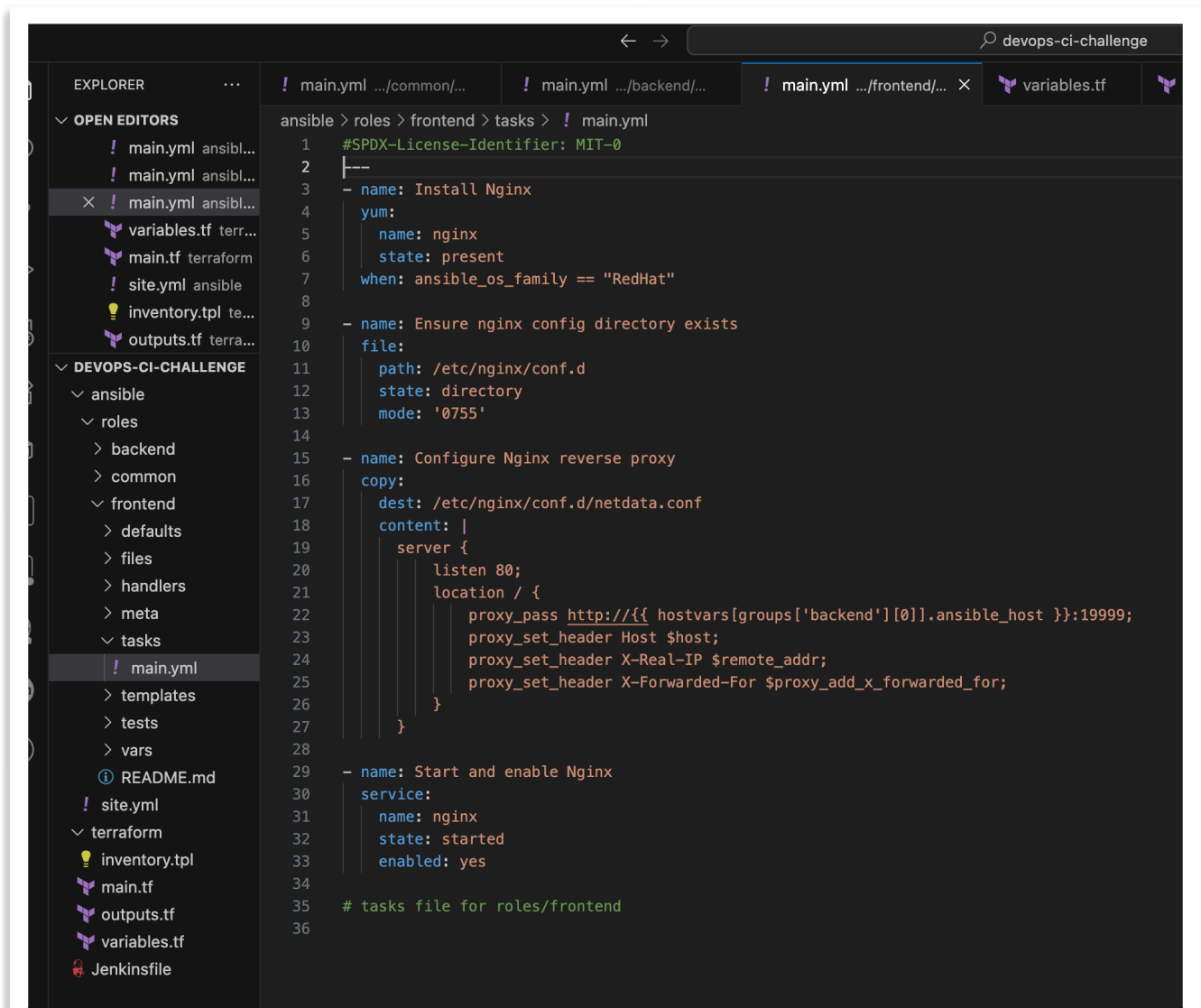
- Install Nginx
- Configure reverse proxy
- Forward **port 80** → **backend:19999**



Open this file in VS Code

Path:

ansible/roles/frontend/tasks/main.yml



Step :11

Jenkinsfile (Terraform + Ansible CI pipeline)

This file will:

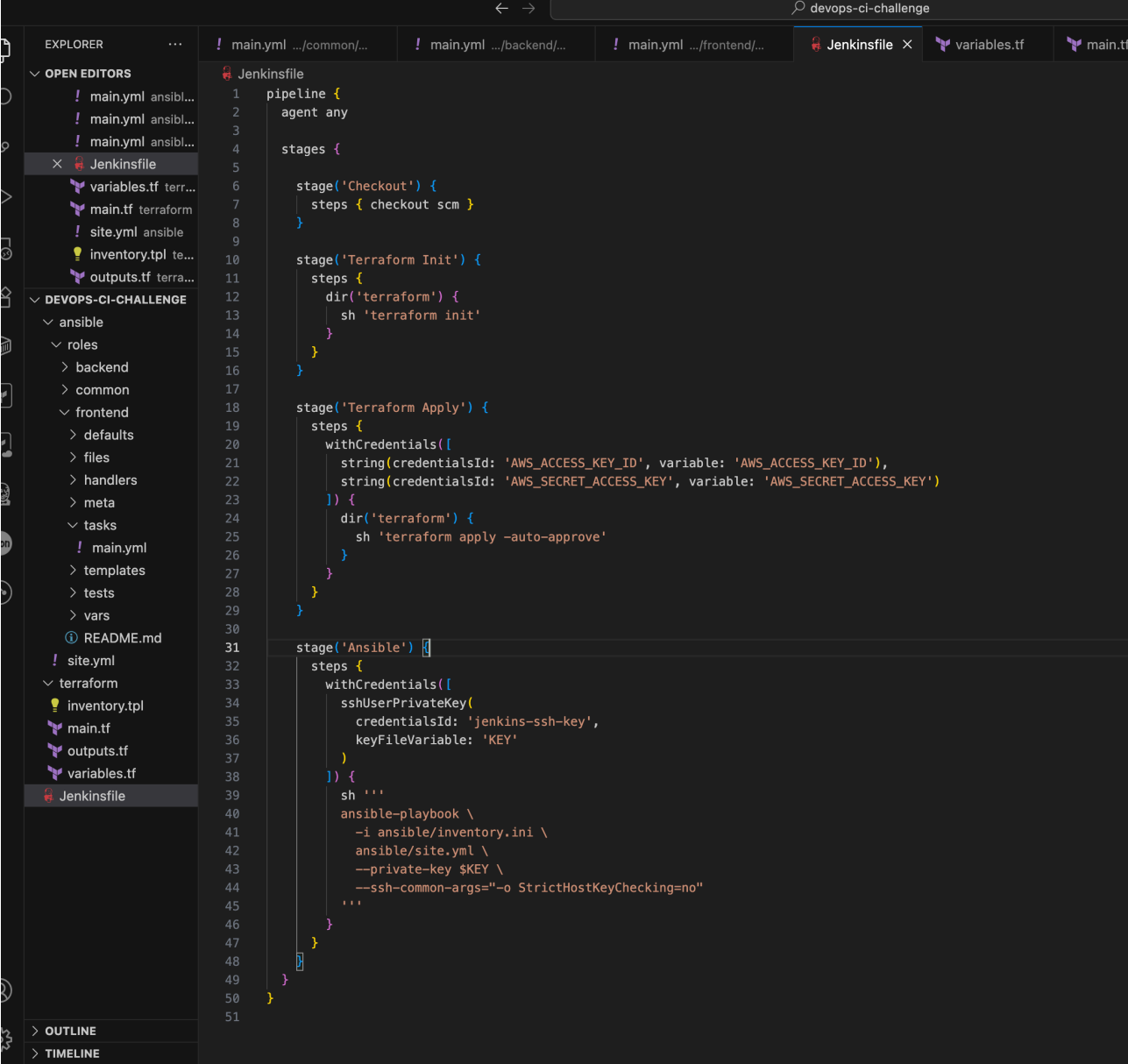
1. Pull your code from GitHub
2. Run **Terraform** (create VMs + inventory)

3. Run **Ansible** (configure frontend & backend)

You are doing this **exactly like your friend**, but now *you* understand it.

Open Jenkinsfile (VS Code)

Path:



```
1 pipeline {
2   agent any
3
4   stages {
5
6     stage('Checkout') {
7       steps { checkout scm }
8     }
9
10    stage('Terraform Init') {
11      steps {
12        dir('terraform') {
13          sh 'terraform init'
14        }
15      }
16    }
17
18    stage('Terraform Apply') {
19      steps {
20        withCredentials([
21          string(credentialsId: 'AWS_ACCESS_KEY_ID', variable: 'AWS_ACCESS_KEY_ID'),
22          string(credentialsId: 'AWS_SECRET_ACCESS_KEY', variable: 'AWS_SECRET_ACCESS_KEY')
23        ]) {
24          dir('terraform') {
25            sh 'terraform apply -auto-approve'
26          }
27        }
28      }
29    }
30
31    stage('Ansible') {
32      steps {
33        withCredentials([
34          sshUserPrivateKey(
35            credentialsId: 'jenkins-ssh-key',
36            keyFileVariable: 'KEY'
37          )
38        ]) {
39          sh '''
40            ansible-playbook \
41              -i ansible/inventory.ini \
42              ansible/site.yml \
43              --private-key $KEY \
44              --ssh-common-args="-o StrictHostKeyChecking=no"
45          '''
46        }
47      }
48    }
49  }
50 }
51
```

step :11

—> now lets push this to the repo

—-> git init

—-> git add .

```
● waseemakram@waseem devops-ci-challenge % pwd
/Users/waseemakram/Desktop/devops-ci-challenge
● waseemakram@waseem devops-ci-challenge % git init

hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /Users/waseemakram/Desktop/devops-ci-challenge/.git/
● waseemakram@waseem devops-ci-challenge % git branch -m main

● waseemakram@waseem devops-ci-challenge % git status

On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Jenkinsfile
    ansible/
    terraform/

nothing added to commit but untracked files present (use "git add" to track)
● waseemakram@waseem devops-ci-challenge % git add .

● waseemakram@waseem devops-ci-challenge % git commit -m "Initial DevOps CI pipeline with Terraform and Ansible"

[main (root-commit) ad203b6] Initial DevOps CI pipeline with Terraform and Ansible
30 files changed, 509 insertions(+)
create mode 100644 Jenkinsfile
create mode 100644 ansible/roles/backend/README.md
create mode 100644 ansible/roles/backend/defaults/main.yml
create mode 100644 ansible/roles/backend/handlers/main.yml
create mode 100644 ansible/roles/backend/meta/main.yml
create mode 100644 ansible/roles/backend/tasks/main.yml
create mode 100644 ansible/roles/backend/tests/inventory
create mode 100644 ansible/roles/backend/tests/test.yml
create mode 100644 ansible/roles/backend/vars/main.yml
create mode 100644 ansible/roles/common/README.md
create mode 100644 ansible/roles/common/defaults/main.yml
create mode 100644 ansible/roles/common/handlers/main.yml
```

—-> git commit -m ""

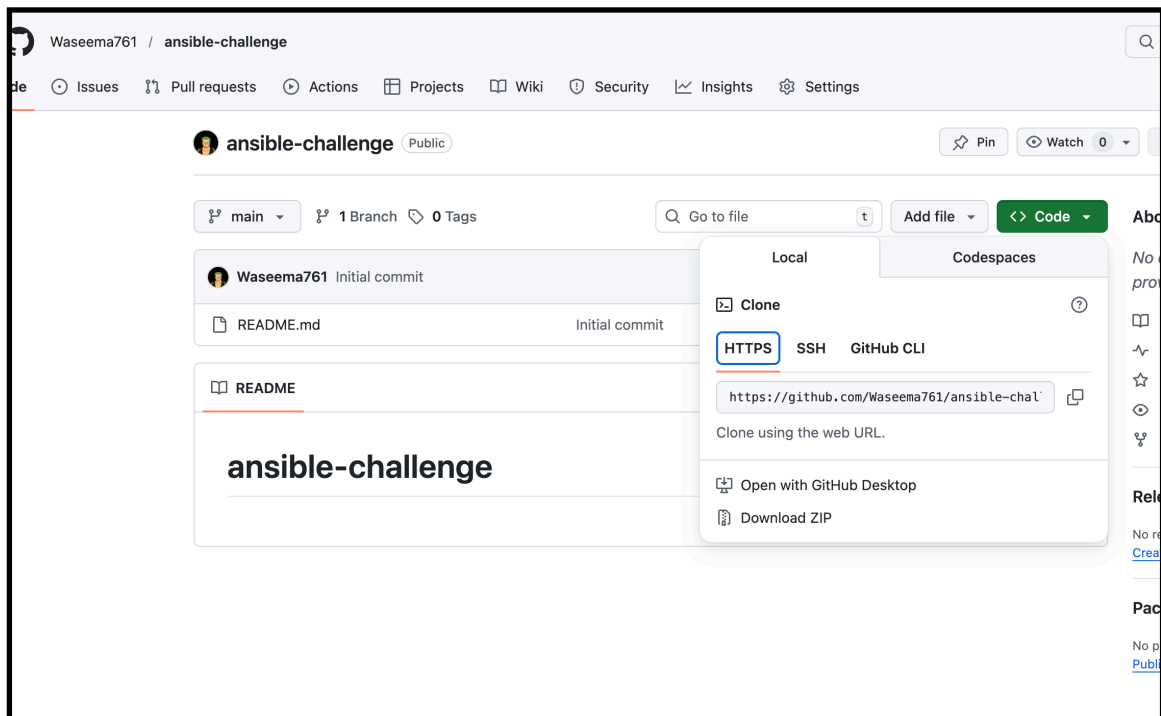
```

create mode 100644 terraform/variables.tf
● waseemakram@waseem devops-ci-challenge % git remote add origin https://github.com/Waseema761/ansible-challenge.git

● waseemakram@waseem devops-ci-challenge % git remote -v
origin https://github.com/Waseema761/ansible-challenge.git (fetch)
origin https://github.com/Waseema761/ansible-challenge.git (push)
○ waseemakram@waseem devops-ci-challenge %

```

—-> git remote add origin httpd: of below



```

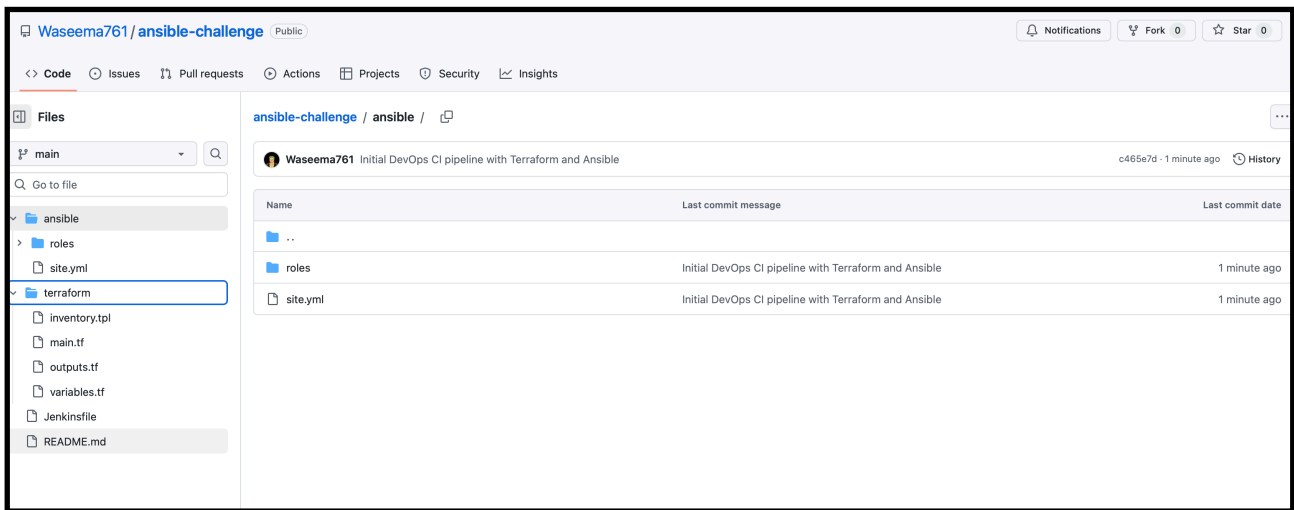
● waseemakram@waseem devops-ci-challenge % git pull origin main --rebase

remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 866 bytes | 173.00 KiB/s, done.
From https://github.com/Waseema761/ansible-challenge
 * branch                main       -> FETCH_HEAD
 * [new branch]          main       -> origin/main
Successfully rebased and updated refs/heads/main.
● waseemakram@waseem devops-ci-challenge % git push -u origin main

Enumerating objects: 49, done.
Counting objects: 100% (49/49), done.
Delta compression using up to 8 threads
Compressing objects: 100% (34/34), done.
Writing objects: 100% (48/48), 6.29 KiB | 3.15 MiB/s, done.
Total 48 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), done.
To https://github.com/Waseema761/ansible-challenge.git
 f41eb2b..c465e7d  main -> main
branch 'main' set up to track 'origin/main'.
○ waseemakram@waseem devops-ci-challenge %

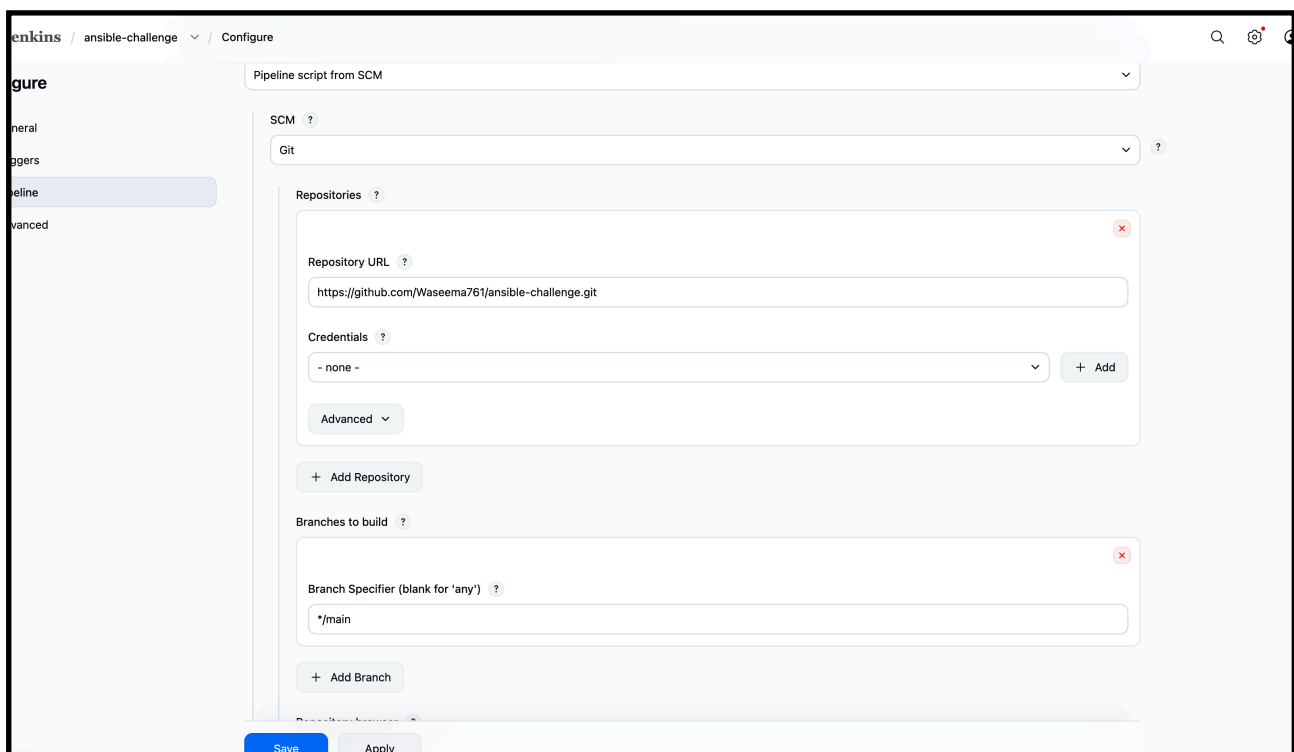
```

--> after pushing



Step :12

--> now open your jenkins gui and create a pipeline job and add this scm url of git hub



—> save

—> and build

Jenkins / ansible-challenge

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

Rename

Pipeline Syntax

Builds > Filter /

Today

- #5 12:45 PM
- #4 12:39 PM
- #3 12:33 PM
- #2 12:30 PM
- #1 12:29 PM

Stage View

Average stage times: (full run time: ~45s)

	Declarative: Checkout SCM	Checkout	Terraform Init	Terraform Apply	Ansible
#5 Feb 04 18:15 1 commit	188ms	155ms	1s	5s	37s
#4 Feb 04 18:09 No Changes	190ms	176ms	1s	5s	31s failed
#3 Feb 04 18:03 1 commit	213ms	172ms	1s	16s	97ms failed
#2 Feb 04 18:00 No Changes	197ms	166ms	1s	4s failed	58ms failed
#1 Feb 04 17:59 No Changes	508ms	402ms	5s	6s failed	317ms failed

Permalinks

- Last build (#4), 6 min 24 sec ago
- Last failed build (#4), 6 min 24 sec ago
- Last unsuccessful build (#4), 6 min 24 sec ago
- Last completed build (#4), 6 min 24 sec ago

Jenkins / ansible-challenge / #5 / Pipeline Overview

Manually run by wassem akram Started 2 min 11 sec ago Queued 1 ms Took 45 sec Changes

Graph

Start Checkout SCM Checkout Terraform Init Terraform Apply Ansible End

Ansible 37s Started 2m 6s ago Jenkins

Checkout SCM 0.16s

Checkout 0.13s

Terraform Init 1.8s

Terraform Apply 5.2s

Ansible 37s

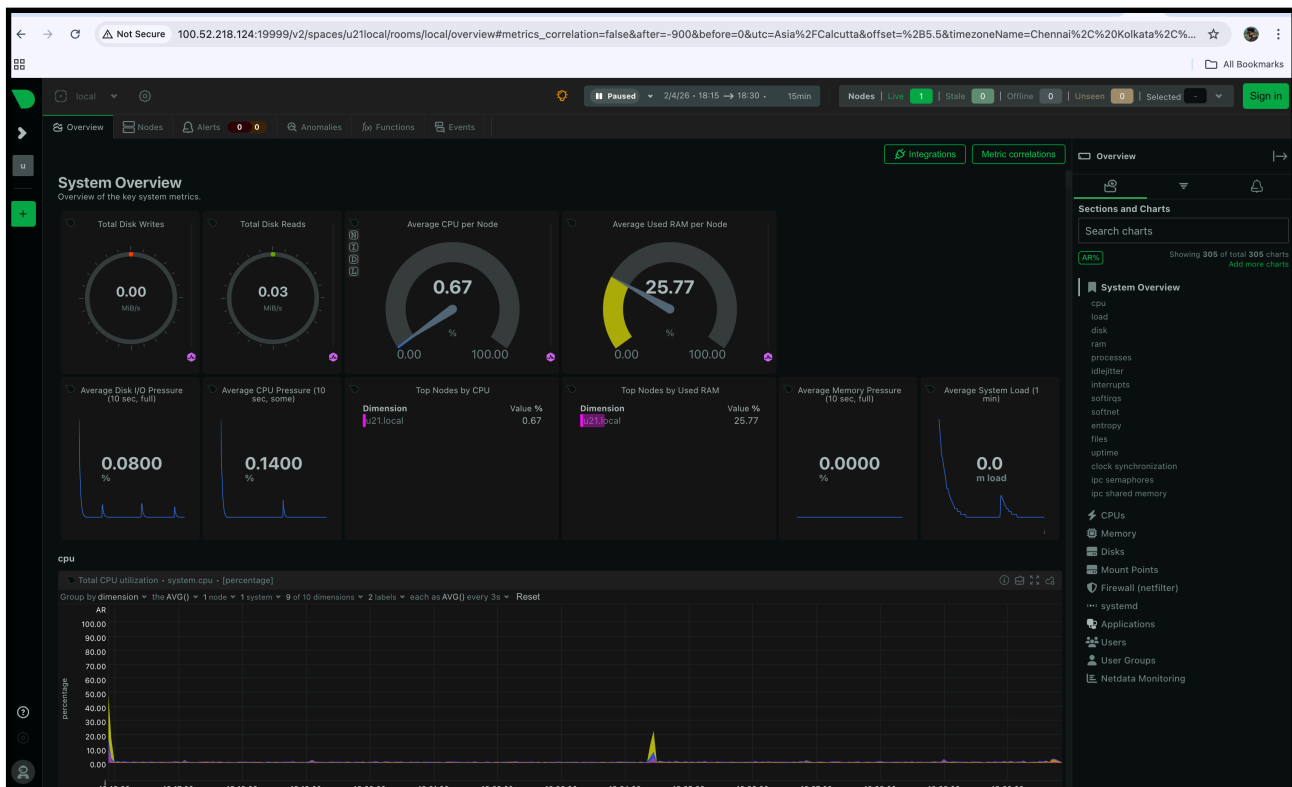
```
ansible-playbook -i ansible/inventory.ini ansible/site.yml --private-key $KEY --ssh-common-args="-o StrictHostKeyChecking=no"
0 + ansible-playbook -i ansible/inventory.ini ansible/site.yml --private-key **** '--ssh-common-args=-o StrictHostKeyChecking=no'
1
2 PLAY [Common configuration on all servers] *****
3
4 TASK [Gathering Facts] *****
5 [WARNING]: Platform linux on host c8.local is using the discovered Python
6 interpreter at /usr/bin/python3.9, but future installation of another Python
7 interpreter could change the meaning of that path. See
8 https://docs.ansible.com/ansible-
9 core/2.15/reference_appendices/interpreter_discovery.html for more information.
```

Verification:

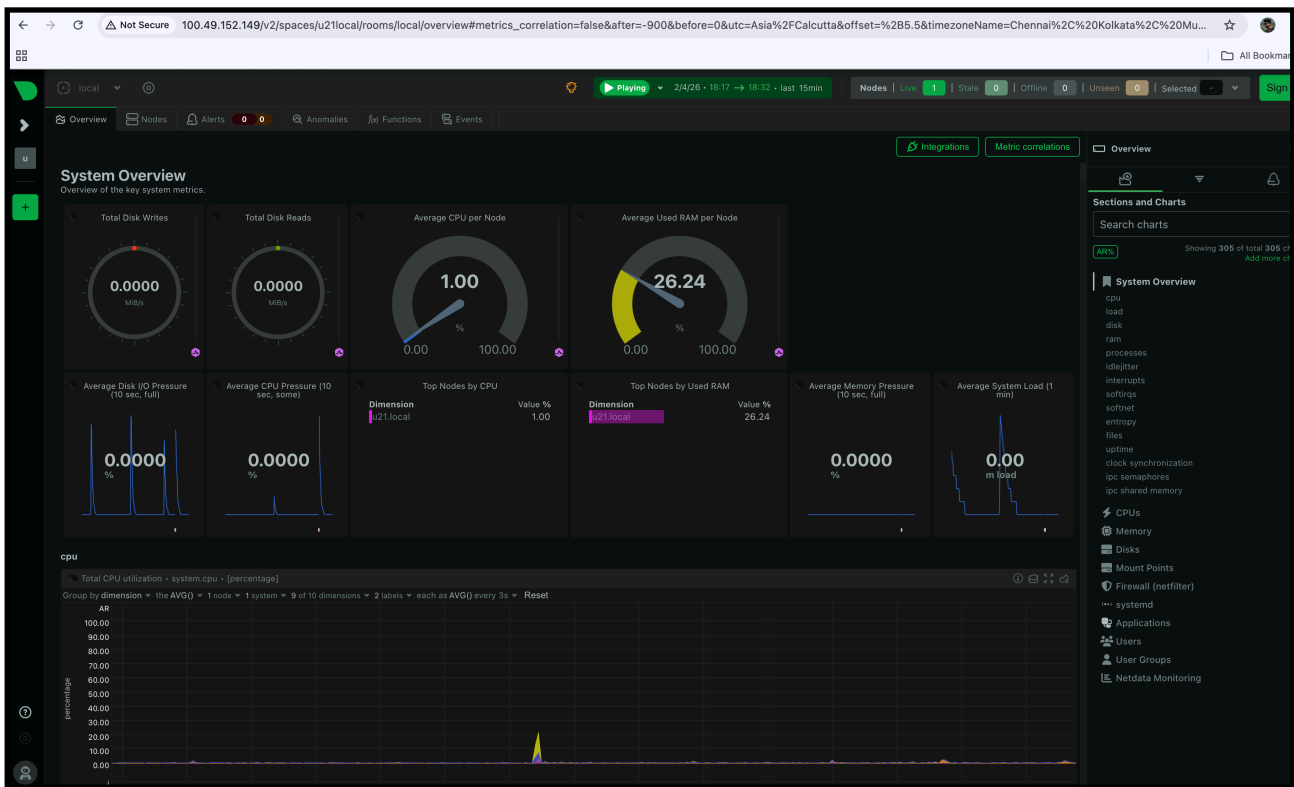
—> check two instances

Instances (1/3) Info								
Find Instance by attribute or tag (case-sensitive)								
All states								
running X Clear filters								
Unselect instance: jenkins-master								
	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 .
☒	jenkins-master	i-0fe26bc99f7f8361b	Running	m7i-flex.large	3/3 checks passed	us-east-1f	ec2-98-93-39-45.comp...	98.93.39.45
☐	u21.local	i-08480595774805471	Running	t3.micro	3/3 checks passed	us-east-1f	ec2-100-52-218-124.co...	100.52.218.1
☐	c8.local	i-0cfb335a349b60eeb	Running	t3.micro	3/3 checks passed	us-east-1f	ec2-100-49-152-149.co...	100.49.152.1

—> backend server u21.local
Ip :100.52.218.124:19999



—> frontend c8.local
ip:100.49.152.149
—> login without using :1999



✓ Proxy works

That means nginx reverse proxy IS WORKING

—> so by this challenge what we have done is
Created CI project structure

You created a proper DevOps repo:

```
devops-ci-challenge/  
├── terraform/  
├── ansible/  
│   └── roles/  
│       ├── common  
│       ├── backend  
│       └── frontend
```

└─ `Jenkinsfile`

✓ Clean separation of infra, config, and CI

2 Terraform – Infrastructure Provisioning

Using Terraform we:

- Created **2 EC2 instances**
 - **Frontend** → `c8.local`
 - **Backend** → `u21.local`
- Used key pair: `jenkins`
- Used **t3.micro (Free Tier)**

Terraform also:

- Generated **Ansible inventory dynamically**
- Output:
 - `frontend_ip`
 - `backend_ip`

👉 Infrastructure is **fully automated**

3 Jenkins – CI Pipeline

In Jenkins we:

- Created **Pipeline from SCM**
- Pulled `Jenkinsfile` from GitHub
- Added credentials:
 - AWS access key & secret
 - SSH private key (`jenkins-ssh-key`)

Pipeline stages:

1. **Checkout code**
2. **Terraform init**
3. **Terraform apply**

4. Ansible playbook execution

✓ One-click pipeline = full deployment

4 Ansible – Configuration Management (Role-based)

◆ Common role (ALL servers)

- Disable SELinux (RedHat)
- Disable firewall (firewalld / ufw)

◆ Backend role (u21.local)

- Install **Netdata**
- Ensure service is running
- Configure Netdata on **port 19999**
- Restart Netdata using handler

Netdata runs **only on backend**

5 Important Security Step (KEY POINT)

Netdata config:

`bind socket to IP = 127.0.0.1`

Means:

- Netdata is **NOT exposed to public**
- Only local access allowed

6 Frontend role (c8.local) – NGINX Reverse Proxy

On frontend:

- Installed nginx
- Configured nginx to:

```
proxy_pass http://backend_ip:19999;
```

nginx listens on **port 80**
Forwards traffic to backend Netdata

7 FINAL RESULT (THE MAGIC PART)

When you opened:

`http://<FRONTEND_PUBLIC_IP>`

✓ You **did NOT** use port 19999

✓ Netdata UI still opened

✓ Metrics visible

—> This **PROVES** nginx reverse proxy is working

ARCHITECTURE FLOW (ONE LINE)

Browser → Frontend (nginx :80) → Backend (Netdata :19999)

—>WHAT THIS PROJECT DEMONSTRATES

- CI/CD with Jenkins
- Infrastructure as Code (Terraform)
- Config management with Ansible roles
- Secure service exposure via reverse proxy
- Real production-style DevOps workflow

